

# Agent 架构三项目小白指南

## 深挖 all-agentic-architectures (35 种 Agent 架构可运行教科书)

来源: X 帖子 @miles\_mazy 《从零学 Agent 架构必看的三个 GitHub 项目》——本指南深挖第 1 个项目并附本机实测实验, 另两个项目做盘点介绍。

主仓库: FareedKhan-dev/all-agentic-architectures (MIT, 4.5k+ star)

35 个架构 · 7 大族

实验: Ollama 本机 Qwen2.5-3B 实测

### 背景: 一条推文, 三个仓库

这条推文面向"想从零学 Agent 架构"的人, 按学习顺序推荐三个互补的项目:

| 顺序          | 仓库                                                       | 一句话                                                                        | 适合                          |
|-------------|----------------------------------------------------------|----------------------------------------------------------------------------|-----------------------------|
| ①<br>先<br>搭 | <a href="#">pguso/ai-agents-from-scratch</a>             | 用 JavaScript + 本地 GGUF 模型, 15 章从"调通一次模型"写到 ReAct / 思维树, 不依赖任何框架            | 想看清每一行代码在干嘛的人               |
| ②<br>再<br>比 | <a href="#">FareedKhan-dev/all-agentic-architectures</a> | 把文献中 35 种 Agent 架构做成统一接口的 Python 库 + 35 份已执行 Notebook + 17 题 Benchmark 排行榜 | 想横向对比"哪种架构治哪种病"的人 (本指南深挖对象) |
| ③<br>后<br>读 | <a href="#">wquguru/harness-books</a>                    | 两本免费书, 分析 Claude Code / Codex 的"挽具 (harness) 工程": 查询循环、工具权限、上下文治理、出错恢复     | 思考"怎么让 Agent 稳定干活"的人        |

### 为什么这三个是"架构学习"的黄金组合?

①教你徒手点火 (知道引擎每个零件), ②给你全车系试驾 (35 种车架同一赛道对比), ③带你进车队维修部 (真实工程怎么让车不掉链子)。三者分别对应"会写 → 会选 → 会养"。

## 疑问清单（目录）

### 第一层：这是什么

- Q1 35 种"Agent 架构"到底在解决什么共同问题？
- Q2 7 大族怎么分类？名字看花眼了。
- Q3 这个库的工程骨架长什么样？（统一 Architecture 合同）

### 第二层：核心机制

- Q4 什么是"确定性选择器模式"？为什么说它是全库的工程纪律？
- Q5 Reflection：生成-评审-改写循环怎么转？
- Q6 Reflection 和 Reflexion 一字之差，差在哪？
- Q7 ReAct 为什么要把 Thought 和 Action 拆成两个节点？
- Q8 Self-Consistency：多采样几次投票，为什么能提升正确率？

### 第三层：家族巡礼

- Q9 规划族（Planning / PEV）和 ReAct 的"边想边做"有何不同？
- Q10 RAG 五变体（Agentic / CRAG / Self-RAG / Adaptive / GraphRAG）各治什么病？
- Q11 记忆族（MemGPT / Voyager）和知识库里的"三层记忆"是什么关系？
- Q12 多智能体族：黑板 / 辩论 / STORM / 元控制器是怎么协作的？

### 第四层：量化与上手

- Q13 Benchmark 是怎么给 35 种架构打分的？78% 意味着什么？
- Q14 没有 API key 能在自己电脑上跑吗？（本机 Ollama 实测路线）
- Q15 三个项目怎么搭配学？推荐路线是什么？

## 逐条解答

### Q1 35 种"Agent 架构"到底在解决什么共同问题？

一句话：**大模型一次生成不可靠，就用"结构"把多次生成组织起来。**

裸调 LLM 只有一问一答，而真实任务需要：多步推理（算 3 步账）、调用工具（查计算器）、自己检查（发现算错了重算）、查资料（RAG）、记住上下文（记忆）、分工协作（多智能体）。35 种架构就是 35 种**组织 LLM 多次调用的"流程图"**——这正是本库用 LangGraph 状态机实现它们的原因：每种架构 = 一张节点+边的图，节点调 LLM 或工具，边决定下一步去哪。



知识库关联：本概念是 **Agent 循环** 笔记的"全集"——那条笔记讲最小两节点环线，本库是把文献中所有变体标准化后的"货架"。

Q2 7 大族怎么分类？

| 族      | 成员                                                                      | 一句话分工                       |
|--------|-------------------------------------------------------------------------|-----------------------------|
| 推理与反思  | Reflection、Reflexion、Self-Discover、CoVe、Constitutional AI               | 让模型 <b>检查并改进</b> 自己的输出      |
| 采样与搜索  | Self-Consistency、Tree of Thoughts、LATS、Mental Loop、Ensemble             | 一条路走不通就 <b>多试几条再挑</b>       |
| 检索 RAG | Agentic RAG、CRAG、Self-RAG、Adaptive RAG、GraphRAG                         | 回答前先 <b>查资料</b> ，且查得越来越聪明   |
| 记忆     | Episodic+Semantic、Graph Memory、MemGPT、Voyager、Agent Workflow Memory     | 跨任务 <b>记住东西</b> （知识库存储外的大脑） |
| 工具与行动  | Tool Use、ReAct、Planning、PEV、SWE-Agent、BrowserAgent                      | 把 LLM 接到 <b>真实世界的手脚</b>     |
| 多智能体   | Multi-Agent、Blackboard、Debate、STORM、Meta-Controller                     | 多个 Agent <b>组队干活</b>        |
| 安全与路由  | Dry-Run、Reflexive Metacognitive、RLHF Self-Improvement、Cellular Automata | <b>护栏</b> 与自我监控             |

记忆口诀：**反思查自己、搜索多开花、RAG 查资料、记忆存本事、工具长手脚、多智能体开班会、安全系安全带。**

Q3 这个库的工程骨架：统一 Architecture 合同

35 个类全部继承同一个抽象基类，只强制实现两个方法，换来三个白得的好处：

```
class Architecture(ABC):
    @abstractmethod
    def build(self) -> CompiledStateGraph: ... # 编译出 LangGraph 状态图
    @abstractmethod
    def run(self, task) -> ArchitectureResult: ... # 跑任务，返回标准化结果
    # 可选: diagram() 自动生成 mermaid 图; explain() 输出原理论述
```

### 统一合同带来三个免费好处

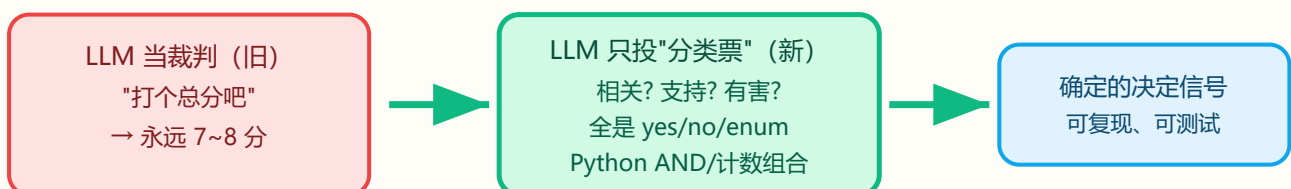
- ① **Notebook 统一**：35 份教程全部同一套调用姿势 `arch.build()` / `arch.run()` / `arch.diagram()`；
- ② **可组合**：元控制器、集成器能把任何架构当黑盒函数调用（“哪个架构都适合被路由”）；
- ③ **文档自动生成**：每篇文档页的图和理论段都出自代码本身，不会和实现脱节。

知识库关联：这就是 **规格书先行 SpecDriven** 的正面案例——先钉死接口合同，35 个实现互不干扰还能互相组合。

### Q4 什么是“确定性选择器模式”？（本库最独特的工程纪律）

很多架构需要 LLM 当“裁判”（这份草稿打几分？这篇检索文档相关吗？）。直接让 LLM 输出“0-10 分”会发现一个病态现象：**LLM 打分挤成一团，几乎什么都是 7-8 分**（flat-band），阈值形同虚设。

本库的解法：**不让 LLM 直接做最终决定，而是让它只回答“分类特征”（布尔/枚举），由 Python 用确定性逻辑组合出决定信号**——这就是确定性选择器模式（deterministic-picker），35 个架构中 13 个使用它，另 9 个靠设计天然免疫。



具体例子（Constitutional AI 架构）：LLM 对宪法每条规则只判 `pass/fail`，Python 做 `all(rules)`，全过才算合格；Crag 对每篇文档判 relevant / ambiguous / irrelevant 三档，Python 按多数档路由。**LLM 出“模糊的判断材料”，Python 下“干脆的决定”**——与知识库 **事实与判断分离**（LLM 只研判不口算）一脉相承。

## Q5 Reflection：生成-评审-改写循环怎么转？

generate 初稿 → critique 评委打分+评语 → 达标？ → refine 带着评语改写 ↺

评委是一个结构化的 LLMJudge：按 Pydantic schema 打 1-10 分并给文字评语；循环出口有两条：  
`score ≥ 目标分` 或 `达到最大迭代数`。注意打分器内部就是 Q4 的确定性选择器——分数由分项布尔/枚举合成，避免"永远 8 分"。

本机实测（Qwen2.5-3B，"给 10 岁小孩解释天为什么是蓝的"）：**首稿即获 8/10，循环 4.7 秒收敛**——评语指出"提到了散射但可更简洁"。

## Q6 Reflection vs Reflexion：一字之差差在哪？

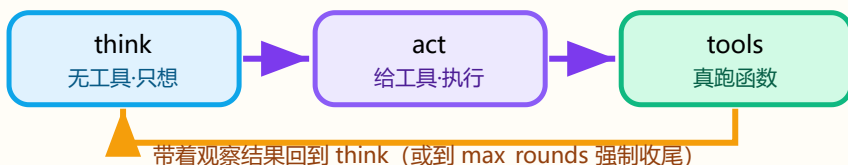
|      | Reflection（反思）      | Reflexion（反身记忆）              |
|------|---------------------|------------------------------|
| 循环单位 | 单任务内：改 <b>这一份草稿</b> | 跨尝试： <b>这次失败的经验</b> 写成文字存进记忆 |
| 灵感来源 | 人类写稿"改二稿"           | 强化学习：失败不丢梯度，丢 <b>语言化的教训</b>  |
| 记忆   | 无（历史只在当次 run 里）     | 情景记忆向量库，下个任务还能检索到            |
| 出口   | 分数达标                | 任务被判成功 / 重试次数耗尽              |

一句话：**Reflection 改"稿"，Reflexion 改"人"**（把教训留给未来的自己）。知识库关联：**本能学习与置信度评分**（失败教训沉淀）与 Reflexion 同思路。

## Q7 ReAct 为什么把 Thought 和 Action 拆成两个节点？

经典 ReAct (ICLR 2023) 是"想一步、动一步"交替：Thought（推理）→ Action（调工具）→ Observation（结果）→ Thought.....但现代工具调用 API 里，Thought 和 Action 挤在同一条消息里，小模型常常**跳过思考直接调工具**，退化成裸 Tool Use。

本库的实现把这个隐患从结构上焊死：**think 节点不给工具，逼模型先输出 Thought；act 节点才给工具做选择**。代价是每轮 2 次 LLM 调用，换来"思考不可跳过"。



本机实测（农夫卖牛奶周收入题）：模型调用 `calculator` 分步算  $15 \times 4 = 60$  升、 $1.2 \times 60 = 72$  元、 $72 \times 7 = 504$  元，**最终答案 504 正确**；但也暴露了 3B 小模型的通病——它反复"确认"已达成的结论不主动收尾，直到 `max_rounds=5` 强制结束。**护栏参数 max\_rounds 就是防止循环不刹车的安全带**。

Q8 Self-Consistency: 多采样几次投票，为什么能提升正确率？

原理像"民主表决"：同一道题，让模型在高温（temperature 0.8，敢犯错也敢出新解）下独立解 5 次，提取每条的最终答案，**Python 多数表决**（确定性选择器）。单个采样有随机错误，但**错误随机、正确收敛**——正确答案像磁铁，错误答案四散，多数表决把磁铁吸出来。论文（Wang et al. 2022）里数学任务提升可达两位数百分点。

本机实测有一个重要发现：3B 小模型做应用题时 5 条采样会**集体偷懒**（全部输出同一个占位答案 "x"），温度 0.8 也拉不开多样性——架构再好也救不了模型能力不足；换单步乘法题后表现见"动手实验"部分。

Q9 规划族（Planning / PEV）和 ReAct 边想边做有何不同？

|      | ReAct        | Planning (Plan-and-Execute) | PEV (Plan-Execute-Verify) |
|------|--------------|-----------------------------|---------------------------|
| 计划时机 | 没有总计划，想一步做一步 | <b>动手前先出完整步骤清单</b>          | 同 Planning                |
| 步骤质检 | 无            | 无（信任每步结果）                   | <b>每步加 Verify，失败带评语重试</b> |
| 适合场景 | 路径不可预知（探索式）  | 任务可分解、工具可靠                  | 工具会失败的真实环境                |

类比：ReAct 是**边走边问路**，Planning 是**出发前先导航**，PEV 是**每个路口核对路牌，错了就地重走**。知识库关联：PEV 的"逐步验证"就是 **质检 Gate 与自我纠错循环** 的 Agent 版。

Q10 RAG 五变体各治什么病？

| 架构           | 治的病                        | 药方                                                                                        |
|--------------|----------------------------|-------------------------------------------------------------------------------------------|
| Agentic RAG  | 普通 RAG 不管要不要都先检索           | 检索变成 <b>工具</b> ，Agent (ReAct 小循环) 自己决定何时查、查什么、查几次                                         |
| CRAG         | 检索回来一堆无关文档照样用              | 每篇文档打分 (相关/模糊/无关, 枚举票)，多数无关就 <b>降级走网络搜索兜底</b>                                             |
| Self-RAG     | 检索时机和文档质量全靠运气              | 模型自发 reflection token: <code>needs_retrieval?</code> 每条文档 relevance/support 由 Python 合成取舍 |
| Adaptive RAG | 简单问题也走全套流程，浪费且慢            | 先给问题 <b>分类定级</b> (免检索/单跳/多跳)，按级路由到不同流程                                                    |
| GraphRAG     | "整部《红楼梦》讲了什么"这类全局问题，向量检索抓瞎 | 抽实体关系建 <b>知识图谱</b> ，按社区聚类+每社区预生成摘要，全局问题读摘要                                                |

知识库关联：基础 RAG 见 **RAG 检索增强生成**；五个变体全用它做底座，本库把这条演进线一次补齐。

**Q11 记忆族 (MemGPT / Voyager) 和"三层记忆"是什么关系?**

知识库已有 **三层记忆** (L1 轨迹→L2 摘要→L3 画像)。本库记忆族的贡献是**管理策略**：  
**MemGPT**把 LLM 上下文比作操作系统内存——Context 是"内存" (有限、FIFO 挤出)，Archival 是"硬盘" (向量库无限)，**Agent 自己决定何时把记忆在内存/硬盘间倒腾** (函数调用式记忆管理)；  
**Voyager** (Minecraft 成名) 更进一步：每次完成任务沉淀一个"技能" (带文档的代码片段) 进技能库，新任务先检索技能库复用，不会就现写——终身学习。  
类比：三层记忆规定了**书放哪几层**，MemGPT 给了**图书管理员** (自己搬书)，Voyager 给了**秘籍收藏夹** (越做越强)。

**Q12 多智能体族：黑板 / 辩论 / STORM / 元控制器怎么协作?**

| 架构                                       | 协作模式                                                             | 类比       |
|------------------------------------------|------------------------------------------------------------------|----------|
| <b>Blackboard</b> (1980s HEARSAY-II 老前辈) | 无中央指挥：所有 Agent 每轮看共享黑板， <b>自评"我能贡献多少"竞价，最自信者上手写</b> ，直到无人再出价     | 头脑风暴会议室  |
| <b>Debate</b>                            | K 轮 × N 个 Agent：每轮各抒己见+点评别人，末轮 <b>多数表决</b>                       | 辩论赛+观众投票 |
| <b>STORM</b>                             | 固定流水线：头脑风暴 N 个视角 → 每视角出题 → 分别作答 → 汇总成文章大纲                        | 编辑部联合采编  |
| <b>Meta-Controller</b>                   | 一个 LLM 路由器看任务选架构：写作走 Reflection、多跳调研走 ReAct、可分解任务走 Planning..... | 分诊台护士    |

知识库关联：与 **多智能体协作** (crewAI 角色分工) 互链——那边是"人事任命"，这边是"协作机制"。

**Q13 Benchmark 怎么给架构打分？78% 意味着什么？**

`benchmarks/run_benchmark.py` 用 17 道固定任务（写作、数学、检索、代码、多跳问答.....）跑所有相关架构，每次运行由 LLM 按 rubric 判对错（确定性选择器式评分），输出排行榜。README 最近成绩：Llama-3.3-70B 跑 33/42 判分点 = **78%**。这个数字的启示不在高低，而在**可横向比较**：同一任务、同一模型、不同架构，谁行谁不行一目了然。它也诚实地标注 "0 mocked runs"——所有示例都是真 LLM 输出，不是摆拍。

**Q14 没有 API key 能在自己电脑上跑吗？**

能。本实验全路线：**Ollama 本地模型 + 本库 ollama provider**。配置就三行（写进 `repo/.env`）：

```
LLM_PROVIDER=ollama
LLM_MODEL=qwen25-3b          # 自己用 GGUF 灌进 ollama 的模型名
OLLAMA_BASE_URL=http://localhost:11434
```

有两个本机实测的坑（详见"动手实验"）：① ollama 官方拉模型时若挂着 Clash 类 fake-ip 代理会报 "redirect target not allowed"，可改从 hf-mirror.com 用 curl 下 GGUF 再 `ollama create`；② 安装别忘了 `networkx`（`architectures/__init__.py` 全量导入 35 个架构，缺一个依赖全体报错）。

**Q15 三个项目怎么搭配学？**

## 推荐路线（推文原顺序 + 本指南补充）

**第一步 搭：** `ai-agents-from-scratch` 按 15 章走一遍（Node.js + 本地模型），重点吃透第 7 章工具调用、第 9 章 ReAct、第 12 章思维树；

**第二步 比：** 用本库把 35 种架构当“试驾”，配合 17 题 benchmark 看排行榜，建立自己的“选型直觉”；

**第三步 读：** `harness-books` 书一第 9 章“十项挽具工程原则” + 书二第 7 章“趋同与分歧”（时间紧只读这两章结论）。

**切忌跳过第一步直接上框架**——框架把循环、工具、记忆都藏进黑盒，出问题只会调提示词碰运气。

## 这个项目给我什么（能力清单）

### 拿到手的能力

- ① **35 个即插即用的 Agent 架构：** `pip install agentic-architectures` 后两行代码调用，换 provider 只改 .env；
- ② **选型直觉：** 7 族分类 + benchmark 排行榜，遇到真实任务能说出“这病该用哪味药”；
- ③ **架构图自动生成：** `arch.diagram()` 输出 mermaid，写文档/汇报直接用；
- ④ **工程纪律范式：** 确定性选择器模式 + 统一合同接口，可直接搬进自己的 Agent 项目；
- ⑤ **35 份真跑 Notebook：** 每个架构的理论都对真实运行输出讲，是最好的进阶教材；
- ⑥ **零 API 费本地复现：** Ollama provider 支持 9 家之外的第 10 种“provider”——自家显卡。

### 我能用它干什么（3-6 个具体用途）

- 1. 给自己的 RAG 应用**快速试装** CRAG / Self-RAG，看纠错值不值得加；
- 2. 写报告前用 **STORM** 生成多视角大纲；
- 3. 用 **Reflection + LLMJudge** 给现有生成管线加“自动二审”；
- 4. 团队内做 Agent 架构**技术分享**（Notebook 直接当幻灯片）；
- 5. 借鉴 Meta-Controller 思路给多技能助手加**任务分诊**。

## 动手实验：一题三架构，本机实测（Qwen2.5-3B）

实验材料在 `exercise\` 目录：`arch_compare.py`（三架构对比）+ `arch_voting_mul.py`（表决补充实验）。全程不花一分钱 API 费，模型跑在本地 Ollama。

## 第 1 步 · 备模型（避开 ollama 拉取被代理拦截的坑）

操作：

```
curl -L -o qwen25-3b-q4_k_m.gguf "https://hf-mirror.com/bartowski/Qwen2.5-3B-Instruct-GGUF"
printf 'FROM ./qwen25-3b-q4_k_m.gguf\nPARAMETER temperature 0\n' > Modelfile
ollama create qwen25-3b -f Modelfile
ollama run qwen25-3b "Reply with exactly: OK"
```

**目的：**ollama 官方源会把下载重定向到被 Clash fake-ip 污染 CDN（报 redirect target not allowed），改从 hf-mirror 拿 GGUF 再灌入。

验证成功标准：

✅ 预期结果：ollama list 出现 qwen25-3b:latest (1.9 GB)；冒烟提问回复 "OK"。

## 第 2 步 · 装库（注意 networkx 坑）

操作：

```
cd repo
python -m venv .venv
.venv\Scripts\pip install -e ".[ollama,faiss]" networkx json-repair
# repo\.env 三行：LLM_PROVIDER=ollama / LLM_MODEL=qwen25-3b / OLLAMA_BASE_URL=http://localhost:11434
.venv\Scripts\python -c "from agentic_architectures.architectures import Reflection, ReAct"
```

**目的：**装 ollama 集成 + FAISS 向量后端；networkx 必须单独补（35 架构全量导入，缺它全体 ImportError）。

✅ 预期结果：打印 imports ok。

### 第 3 步 · 跑三架构对比

操作：

```
cd exercise
PYTHONUTF8=1 ../repo/.venv/Scripts/python arch_compare.py
```

**目的：**Reflection 写科普、ReAct 算农夫卖奶账（自带 calculator 工具）、Self-Consistency 解应用题并和单次直答对比。

**真实结果摘要**（完整输出在 `exercise/results.txt`）：

#### Reflection: 4.7 秒收敛

首稿评委打 **8/10**（目标分 8），评语：“面向 10 岁儿童的解释清晰有趣，但可更简洁精确”——一次达标直接收官，这就是“达标即停”循环。

#### ReAct: 24 步、3 次工具调用，答案 504 正确

轨迹里能看到显式分工：think 节点出 Thought（“先算日产奶量”），act 节点调

`calculator("15*4")` → 60, `calculator("1.2*60")` → 72,

`calculator("72*7")` → 504。也看到小模型通病：得出答案后仍反复“确认”不收尾，直到 `max_rounds=5` 强制结束——**护栏参数救了场**。

#### Self-Consistency: 应用题翻车现场（诚实的教训）

应用题“ $24 \times 7 - 30 - 12 = ?$ ”标准答案 126：单次直答答“22”（错），5 票表决集体偷懒输出占位符“x”（也错）。**5 条采样一字不差**——温度 0.8 在 3B 小模型上拉不开多样性，长推理链是它能力边界。**架构不是魔法，模型能力是地基**。

#### 第 4 步 · 补充实验：换短推理链的乘法题再投票

操作： `PYTHONUTF8=1 ../repo/.venv/Scripts/python arch_voting_mul.py` (两轮：第一轮 5 道两位数乘法、`num_predict=300`；第二轮换 5 道**更难**的乘法、`num_predict=900`)

真实结果摘要 (完整输出在 `exercise\results.txt` 与 `exercise\mul_console2.log`)：

##### 简单乘法 ( $17 \times 23$ 等)：直答 5/5，表决却因“截断”翻车

单次直答全对；但 5 票表决只提取出 1 票有效答案。根因：`num_predict` 封顶 300 把推理过程截断，JSON 解析失败——**答案提取依赖结构化输出，截断=报废**。

##### 边界乘法 ( $87 \times 69$ 等)：直答 0/5，表决 0/5——但错法不同

5 道题在 3B 模型的能力边界上：单次直答 0/5，错得“像模像样” (5853/7104/3664...都是接近的错数)；高温采样时 ollama 的结构化输出回退彻底失控——采样里混进正则碎片、PHP 代码、HTML 标签，**乱码还能赢得多数票**。这正是库自己能力矩阵里标注的 `ollama structured_output: False` 的代价。

**实验总教训 (比成功更值钱)**：①架构救不了模型能力地板——Self-Consistency 的前提是“错误随机、正确收敛”，3B 模型在难题上错误是**系统性的**，投票无效；②provider 能力矩阵是硬约束——结构化输出不可靠时，依赖它的架构 (采样搜索族、部分 RAG 族) 会静默劣化；③README 的 78% 是 Llama-3.3-**70B** 的成绩，本地小模型复现应定位在“学机制”而非“追分数”。

✅ 预期结果：`results.txt` / `mul_console2.log` 末尾出现“汇总”行；能观察到 tally 分布与两种方式的正确数对比——即使是翻车，也是真实的、可复现的翻车。

## 第 5 步 · 生成架构图 (白得的文档素材)

**操作:** `print(Reflection().diagram())` 等——每个架构自动吐 mermaid 状态图。

**目的:** 把"流程图"从代码里直接抠出来, 写汇报不用手画。

✓ **预期结果:** 输出 mermaid 文本 (graph TD ...), 粘进任何支持 mermaid 的文档即渲染。

## 另两项目盘点 (按推文顺序①③)

### ① ai-agents-from-scratch (pguso, JS, 4.8k star)

**是什么:** 15 章渐进教程, Node.js + node-llama-cpp 跑本地 GGUF 模型, **零框架徒手搭 Agent**; 每章 = 可运行代码 + 逐行讲解 CODE.md + 概念 CONCEPT.md; 另有配套网站与 Python 姊妹版 (pguso/agents-from-scratch)。

**章节主线:** 调通模型 → 系统提示词 → 函数调用 (工具) → 持久记忆 → **ReAct 循环** → 思维树/思维图搜索 → 可审计决策 → embedding 工具路由。

**什么时候用:** 觉得框架太黑盒、想每一行都懂的时候。**环境:** Node 18+, 8-16GB 内存, 自下一个 GGUF 模型。

### ③ harness-books (wquguru, 免费书两本, 3.2k star)

**是什么:** 不写"怎么调模型", 写"怎么让模型**持续、有界、可问责**地干活"的挽具工程。书一《Harness Engineering: A Design Guide to Claude Code》9 章: 提示词即控制面、查询循环 (Agent 系统的心跳)、工具与权限、上下文治理 (CLAUDE.md/Compact)、错误恢复、多智能体验证、团队落地、十项原则; 书二对比 Claude Code 与 Codex 两种挽具哲学的趋同与分歧。

**与知识库的关系:** 本书是 **AgentHarness 智能体挽具、ClaudeCodeHooks 与 fail-open、上下文预算与战略压缩** 等笔记的"原著级"展开——vault 里已学的 harness 概念这里成体系。时间紧只读书一第 9 章 + 书二第 7 章 (作者自己标的速读路线)。